

A SMALL THEME FOR MODERN SLIDES

PostgreSQL for Odoo

SQL fluency, roles, indexes, and backup discipline

Weblearns Academy
weblearns@training.local
Odoo Full-Stack Developer Program
Lecture materials / July 2026



Agenda

- 1 Install, roles, grants, and configuration
- 2 DDL, CRUD, joins, and aggregations
- 3 Indexes, EXPLAIN, and lock awareness
- 4 Dump, restore, and Odoo DB habits

TOC

1 A theme for modern talks

How this deck is taught

AWESOME-STYLE FRAME

Number rails, angled title art, and section slides are built in.

METROPOLIS DEFAULTS

Progress bars, block styles, and fonts are set from theme options.

DENSE TEACHING

Two-column frames pack objectives, examples, and pitfalls.

PostgreSQL

SQL

Indexes

Backup

1 DB01 – DB05

PostgreSQL for Odoo · teaching block 1

DB01 Install PostgreSQL on Ubuntu

Objectives, key points, and lab target

Objectives

- Install PostgreSQL packages on Ubuntu in a repeatable way.
- Understand the system service, default cluster, and postgres OS user.
- Verify readiness before connecting Odoo to the server.

Key points

- Use apt for predictable installs and security updates.
- The default service is usually postgresql, with one or more versioned clusters behind it.
- The postgres OS account is for database administration, not for running Odoo.
- Always verify the server with psql before blaming Odoo.

Lab target

Install PostgreSQL on a fresh Ubuntu machine, confirm the service is active, and capture the output of `SELECT version()`.

DB01 Install PostgreSQL on Ubuntu

Teaching notes

A clean PostgreSQL installation starts with knowing which package source you trust. Ubuntu ships PostgreSQL packages that are stable and easy to patch, while the upstream PostgreSQL APT repository gives access to newer major versions sooner.

On Ubuntu, PostgreSQL is managed as a systemd service and normally creates a default cluster during installation. The cluster is a data directory plus configuration files, not just a running process.

The postgres Linux user is intentionally separate from your application user. It owns the database files and can connect locally as the postgres database superuser through peer authentication.

Also remember

- For Odoo training and development, keep the database server local unless there is a clear operational reason to...
- After installation, verify the service state, listening port, and psql connection before installing Odoo. Debugging...

DB01 Install PostgreSQL on Ubuntu

Example and pitfalls

Ubuntu package installation

This installs PostgreSQL, starts the service, and proves that the local postgres administrator can execute SQL.

```
sudo apt update
sudo apt install -y postgresql postgresql-contrib
sudo systemctl enable --now postgresql
systemctl status postgresql --no-pager
sudo -u postgres psql -c "SELECT version();"
```

Common mistakes

- Installing PostgreSQL but never checking whether the service is actually running.
- Running Odoo as the postgres Linux user instead of creating a dedicated Odoo database...
- Mixing Ubuntu packages and upstream packages without documenting the version source.
- Opening PostgreSQL to the network before authentication rules and firewall rules are...

DB01 Install PostgreSQL on Ubuntu

Extra patterns

Install checks

-

DB02 psql CLI, Users, Roles, and Grants

Objectives, key points, and lab target

Objectives

- Connect with psql using local and password authentication.
- Create roles suitable for Odoo development.
- Grant only the privileges an application role needs.

Key points

- A PostgreSQL role is the central identity and privilege unit.
- LOGIN makes a role usable as a user account.
- CREATEDB is convenient for Odoo development but should be reviewed for production.
- psql meta commands are part of a practical database debugging workflow.

Lab target

Create a role named `odoo17_lab` with CREATEDB, connect with psql over 127.0.0.1, and list all roles with `\du`.

DB02 psql CLI, Users, Roles, and Grants

Teaching notes

psql is the fastest diagnostic tool for PostgreSQL because it removes the application layer from the problem. A developer who can inspect roles, databases, schemas, and locks from psql can solve most Odoo database issues directly.

PostgreSQL uses roles for both users and groups. A role can log in when it has LOGIN, and it can own objects, inherit privileges, or be granted to other roles depending on how you design access.

Odoo usually connects with one role that owns or can create its databases in development. In production, many teams restrict the role to a specific database and avoid broad CREATEDB permissions after deployment.

Also remember

- Grants are scoped to databases, schemas, tables, sequences, and functions. Granting a database privilege does not...
- Password authentication requires both a password on the role and a pg_hba.conf rule that permits password-based...

DB02psql CLI, Users, Roles, and Grants

Example and pitfalls

Create an Odoo database role

This creates a login role that can create databases, then confirms that password authentication works over TCP.

```
sudo -u postgres createuser --createdb --pwprompt odoo  
psql -h 127.0.0.1 -U odoo17 -d postgres
```

```
-- Inside psql  
\conninfo  
\du  
SELECT current_user;
```

Common mistakes

- Assuming a database grant automatically grants privileges on every table.
- Forgetting that peer authentication uses the Linux user name, not the typed database...
- Giving SUPERUSER to application roles to avoid learning the real permission issue.
- Leaving training passwords in shell history or shared notes.

Grant schema privileges

```
CREATE DATABASE training_owner OWNER postgres;  
CREATE ROLE training_app LOGIN PASSWORD 'change-me';  
\c training_owner  
GRANT USAGE, CREATE ON SCHEMA public TO training_app;  
ALTER DEFAULT PRIVILEGES IN SCHEMA public  
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO training_app;
```

DB03 Create and Manage Databases

Objectives, key points, and lab target

Objectives

- Create databases with the correct owner and encoding.
- Understand templates, connection limits, and database ownership.
- Prepare databases for Odoo import and development workflows.

Key points

- One Odoo database maps to one Odoo instance dataset.
- Use explicit owners for predictable module installation and upgrades.
- UTF8 is expected for modern Odoo deployments.
- Do not drop or rename databases casually in shared environments.

Lab target

Create two training databases owned by your Odoo role, list them with \l, and remove one after confirming its owner.

DB03 Create and Manage Databases

Teaching notes

A PostgreSQL database is an isolation boundary for Odoo data. Each Odoo database contains its own registry metadata, business records, installed modules, and attachments references.

The database owner matters because Odoo creates and updates many database objects during module installation and upgrades. If the role lacks ownership or enough privileges, upgrade failures can appear deep inside ORM operations.

UTF8 encoding is the normal choice for Odoo. Training environments should make this explicit so later collation, translation, and text search behavior are easier to explain.

Also remember

- Template databases allow PostgreSQL to copy an existing database structure at creation time. For ordinary Odoo work,...
- Connection limits can protect a shared database server from one workload consuming every backend process. For local...

DB03 Create and Manage Databases

Example and pitfalls

Create databases from SQL and CLI

The CLI is concise for daily work, while SQL makes ownership, encoding, template, and limits explicit.

```
createdb -h 127.0.0.1 -U odoo17 --encoding=UTF8 odoo17

psql -U postgres -d postgres -c "
CREATE DATABASE odoo17_reporting
    WITH OWNER odoo17
    ENCODING 'UTF8'
    TEMPLATE template0
    CONNECTION LIMIT 50;
"
```

Common mistakes

- Creating an Odoo database owned by postgres and then running Odoo as a different...
- Using mixed encodings or locale settings across environments without documenting them.
- Dropping a database from the wrong terminal session.
- Copying a database while users or cron jobs are still writing to it.

DB03 Create and Manage Databases

Extra patterns

Odoo habit

Name development databases clearly, for example `odoo17_dev_customer_a`, so backups and logs are easier to match.

DB04 PostgreSQL Data Types for Odoo Developers

Objectives, key points, and lab target

Objectives

- Recognize PostgreSQL types commonly produced by Odoo fields.
- Choose SQL types for reporting tables and integration staging tables.
- Avoid type choices that damage money, date, or JSON accuracy.

Key points

- Understand the SQL type behind each important Odoo field.
- Use numeric for exact business decimals.
- Datetimes are stored in UTC by Odoo.
- JSONB is useful, but relational data still belongs in relations.

Lab target

Inspect five columns from res_partner or another Odoo table and map each SQL type back to the likely Odoo field type.

DB04 PostgreSQL Data Types for Odoo Developers

Teaching notes

Odoo hides most column definitions behind its ORM, but PostgreSQL types still affect performance, correctness, and reporting. Senior Odoo developers should be able to inspect a table and understand what the ORM generated.

Text-like fields commonly map to varchar or text. The practical distinction is less about storage and more about validation, indexing, and how the Odoo field communicates intent to users.

Numeric choices matter for money and quantities. Use numeric for exact decimal values when precision matters, and avoid float for values that must reconcile financially.

Also remember

- Date and timestamp columns support business workflows such as accounting periods, scheduled actions, and SLA...
- JSONB is useful for flexible integration payloads, analytic properties, and denormalized metadata. It is powerful, but...

DB04 PostgreSQL Data Types for Odoo Developers

Example and pitfalls

Inspect generated Odoo column types

The query teaches developers to inspect real Odoo tables, while the staging table shows conservative type choices for integrations.

```
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'res_partner'
ORDER BY ordinal_position;

CREATE TABLE integration_payload (
    id bigserial PRIMARY KEY,
    external_id varchar NOT NULL,
    amount numeric(16, 2) NOT NULL,
    received_at timestamp without time zone NOT NULL,
# ...
```

Common mistakes

- Using float for money because it looks simpler in test data.
- Storing dates as text and then losing reliable filtering and sorting.
- Putting frequently joined business identifiers only inside JSONB.
- Assuming Odoo user timezone conversion changes how datetimes are stored in PostgreSQL.

DB05 DDL: CREATE, ALTER, and DROP

Objectives, key points, and lab target

Objectives

- Use DDL safely in development and maintenance tasks.
- Understand how DDL locks affect live Odoo systems.
- Plan schema changes with reversibility and backups in mind.

Key points

- Let Odoo module definitions manage normal model tables.
- DDL can block live application traffic.
- Review ALTER operations for rewrite or scan behavior.
- Treat DROP as a backup-required operation.

Lab target

Create a reporting schema and table, alter it with one new column, inspect it with `\d`, then drop it after confirming the object name.

DB05 DDL: CREATE, ALTER, and DROP

Teaching notes

Data Definition Language changes the structure of the database. In Odoo, most application schema changes should come from module code and migrations, not from manual SQL typed into production.

CREATE statements add tables, indexes, schemas, views, and functions. They are safe when planned, but object names should follow clear conventions so DBAs and developers can identify ownership.

ALTER statements can be cheap or expensive depending on the operation and PostgreSQL version. Adding a nullable column is usually fast, while changing a type or setting a not-null constraint may scan or rewrite data.

Also remember

- DROP statements are final from the perspective of the current transaction once committed. A backup is not optional...
- DDL takes locks, and those locks can block Odoo workers. Even short DDL can create visible downtime if it waits behind...

DB05 DDL: CREATE, ALTER, and DROP

Example and pitfalls

DDL for a reporting helper table

This illustrates common DDL statements in a non-Odoo schema where manual SQL may be appropriate.

```
CREATE SCHEMA IF NOT EXISTS reporting;

CREATE TABLE reporting.daily_sales_snapshot (
    snapshot_date date PRIMARY KEY,
    order_count integer NOT NULL DEFAULT 0,
    untaxed_total numeric(16, 2) NOT NULL DEFAULT 0
);

ALTER TABLE reporting.daily_sales_snapshot
    ADD COLUMN company_id integer;
# ...
```

Common mistakes

- Manually altering Odoo-owned tables and then being surprised when module upgrades...
- Running ALTER TABLE during business hours without checking locks and table size.
- Dropping a column before confirming whether computed fields, reports, or integrations...
- Leaving experimental tables in public schema with unclear ownership.

2 DB06 – DB10

PostgreSQL for Odoo · teaching block 2

DB06 Primary Keys, Foreign Keys, and Constraints

Objectives, key points, and lab target

Objectives

- Explain how constraints protect relational integrity.
- Use primary keys, foreign keys, unique constraints, and checks.
- Connect database constraints to Odoo model design.

Key points

- Constraints are stronger than application-only validation.
- Unique constraints are important under concurrent writes.
- Foreign keys keep relational references honest.
- Use SQL constraints for invariants that must never be violated.

Lab target

Build two related tables with a foreign key and a unique constraint, then attempt inserts that prove each constraint works.

DB06 Primary Keys, Foreign Keys, and Constraints

Teaching notes

Constraints are the database's way of refusing impossible data. They remain active regardless of whether data is written by Odoo, an import script, psql, or an integration job.

A primary key uniquely identifies each row and is the target for many relationships. Odoo models normally use an integer id primary key generated by the database.

Foreign keys preserve references between tables. In Odoo, Many2one fields often create foreign keys unless the field definition or module history prevents it.

Also remember

- Unique constraints prevent duplicates that application checks can miss under concurrency. For example, two users...
- Check constraints enforce local row rules such as positive quantities or valid ranges. They are simple, fast, and...

DB06 Primary Keys, Foreign Keys, and Constraints

Example and pitfalls

Relational constraints

The schema protects identity, references, positive amounts, and invoice uniqueness even if data is loaded outside Odoo.

```
CREATE TABLE training_customer (  
    id bigserial PRIMARY KEY,  
    email varchar NOT NULL UNIQUE,  
    credit_limit numeric(16, 2) NOT NULL DEFAULT 0,  
    CHECK (credit_limit >= 0)  
);  
  
CREATE TABLE training_invoice (  
    id bigserial PRIMARY KEY,  
    customer_id bigint NOT NULL REFERENCES training_customer(id) ON DELETE RESTRICT,  
    # ...
```

Common mistakes

- Relying only on Python validation for uniqueness in concurrent workflows.
- Using ON DELETE CASCADE without considering accounting and audit requirements.
- Adding constraints to dirty data without first finding violations.
- Creating nullable foreign keys when the business relationship is actually required.

DB07 CRUD: INSERT, SELECT, UPDATE, and DELETE

Objectives, key points, and lab target

Objectives

- Write basic CRUD SQL confidently.
- Understand transactions and RETURNING clauses.
- Relate direct SQL changes to Odoo ORM safety rules.

Key points

- Always write SELECT first to confirm the target rows before UPDATE or DELETE.
- RETURNING is useful for inserts and updates.
- Direct SQL bypasses Odoo ORM behavior.
- Use transactions for safe experimentation.

Lab target

Insert a row in a transaction, update it with RETURNING, verify the result, and roll back so the table returns to its original state.

DB07 CRUD: INSERT, SELECT, UPDATE, and DELETE

Teaching notes

CRUD operations are the foundation of every database-backed application. Even when Odoo's ORM writes SQL for you, understanding the underlying statements helps you debug logs and performance plans.

INSERT creates rows and can return generated values such as id. RETURNING is especially useful in scripts because it avoids a second query to discover what was inserted.

SELECT retrieves data and should be shaped intentionally. Pulling every column from a large Odoo table is rarely the right diagnostic habit when you only need ids, names, and a status field.

Also remember

- UPDATE changes existing rows and is dangerous without a precise WHERE clause. In Odoo databases, direct UPDATE can...
- DELETE removes rows and should be used with even more caution. Many Odoo records should be archived with active =...

DB07 CRUD: INSERT, SELECT, UPDATE, and DELETE

Example and pitfalls

Safe CRUD workflow

The transaction proves the statements and then rolls them back, which is a safe habit when learning or diagnosing.

```
BEGIN;  
  
INSERT INTO training_customer (email, credit_limit)  
VALUES ('ada@example.com', 5000.00)  
RETURNING id, email;  
  
SELECT id, email, credit_limit  
FROM training_customer  
WHERE email = 'ada@example.com';  
  
# ...
```

Common mistakes

- Running UPDATE or DELETE without a WHERE clause.
- Changing Odoo data directly and expecting ORM computed fields or tracking to run.
- Using SELECT * on large tables during production incidents.
- Forgetting to COMMIT a successful transaction or ROLLBACK a test transaction.

DB08 Filtering with WHERE, LIKE, ILIKE, IN, and BETWEEN

Objectives, key points, and lab target

Objectives

- Use common filtering operators precisely.
- Understand case-sensitive and case-insensitive pattern matching.
- Avoid filters that defeat indexes or return unintended records.

Key points

- LIKE is case-sensitive; ILIKE is case-insensitive.
- BETWEEN includes both ends of the range.
- Use IS NULL for missing optional values.
- Index-friendly filters matter on large Odoo tables.

Lab target

Write three SELECT queries against an Odoo database: one with ILIKE, one with IN, and one date range using >= and <.

DB08 Filtering with WHERE, LIKE, ILIKE, IN, and BETWEEN

Teaching notes

Filtering is where SQL becomes practical for support work. Good WHERE clauses narrow the problem to the exact company, date range, state, or external identifier you need to investigate.

LIKE is case-sensitive pattern matching in PostgreSQL, while ILIKE is case-insensitive. ILIKE is convenient for support searches but can be slower unless supported by the right index strategy.

IN is clearer than a long chain of OR comparisons when matching a known set of values. It is common in Odoo diagnostics because record ids are often copied from logs, URLs, or failed queue jobs.

Also remember

- BETWEEN includes both boundaries. That is useful for dates when you mean a closed range, but for datetimes many teams...
- NULL requires IS NULL or IS NOT NULL, not equals comparisons. This distinction matters in Odoo because optional...

DB08 Filtering with WHERE, LIKE, ILIKE, IN, and BETWEEN

Example and pitfalls

Odoo-style support filters

The examples use practical filters for partners and orders while avoiding an ambiguous datetime BETWEEN boundary.

```
SELECT id, name, email
FROM res_partner
WHERE active IS TRUE
      AND email ILIKE '%@example.com'
      AND company_id IN (1, 3, 7)
ORDER BY name
LIMIT 50;

SELECT id, name, date_order
FROM sale_order
# ...
```

Common mistakes

- Expecting LIKE to be case-insensitive.
- Using BETWEEN with datetimes and accidentally including or excluding boundary records.
- Writing column = NULL instead of column IS NULL.
- Starting every text search with % and then wondering why indexes are not used.

DB09 Joins for Relational Odoo Data

Objectives, key points, and lab target

Objectives

- Choose INNER JOIN and LEFT JOIN correctly.
 - Trace Odoo Many2one and Many2many relationships in SQL.
 - Read joined results without multiplying rows accidentally.
- ## Key points
- INNER JOIN filters to matching rows.
 - LEFT JOIN preserves unmatched left-side rows.
 - Many2many joins require a relation table.
 - One-to-many joins can duplicate parent values in result sets.

Lab target

Join sale_order to res_partner, then write a separate query that follows one Many2many relation table in your database.

DB09 Joins for Relational Odoo Data

Teaching notes

Joins combine rows from related tables, which is essential in Odoo because business data is highly relational. A sale order links to partners, companies, currencies, users, order lines, stock moves, invoices, and more.

INNER JOIN returns rows only when both sides match. It is appropriate when the relationship is required and missing related data would indicate invalid or irrelevant records.

LEFT JOIN keeps the left-side row even when the related row is missing. This is useful for optional fields, incomplete setup, and diagnostics where missing data is part of the question.

Also remember

- Many2one relationships are usually simple joins from a foreign key column to the related table's id. Many2many...
- One-to-many joins can multiply rows because each parent may have many children. That is expected, but it becomes a bug...

DB09 Joins for Relational Odoo Data

Example and pitfalls

Join sale orders to customers and tags

The first query follows Many2one fields, and the second shows the relation table pattern behind a partner tag Many2many.

```
SELECT so.id, so.name, rp.name AS customer, rc.name AS company
FROM sale_order so
JOIN res_partner rp ON rp.id = so.partner_id
LEFT JOIN res_company rc ON rc.id = so.company_id
WHERE so.state IN ('sale', 'done')
ORDER BY so.date_order DESC
LIMIT 20;
```

```
SELECT rp.name, category.name AS tag
FROM res_partner rp
# ...
```

Common mistakes

- Using INNER JOIN where optional data requires LEFT JOIN.
- Summing parent totals after joining multiple child tables.
- Guessing Many2many relation table names instead of inspecting the model or database.
- Joining on display names instead of stable ids.

DB10 GROUP BY, HAVING, and Aggregates

Objectives, key points, and lab target

Objectives

- Summarize Odoo data with aggregate functions.
- Use GROUP BY and HAVING in the correct order.
- Avoid aggregation mistakes caused by joins and NULL values.

Key points

- GROUP BY sets the result grain.
- HAVING filters aggregate results.
- COUNT(*) and COUNT(column) answer different questions.
- Pre-aggregate child tables before joining multiple one-to-many relationships.

Lab target

Create a grouped sales or invoice query with COUNT and SUM, then add a HAVING clause that filters low-value groups.

DB10 GROUP BY, HAVING, and Aggregates

Teaching notes

Aggregates turn detail records into business signals such as totals, counts, minimums, maximums, and averages. Odoo dashboards and reports often reduce to the same concepts expressed through ORM `read_group` or SQL.

GROUP BY defines the grain of the result. If you group sales by company and month, every selected non-aggregate column must belong to that grain.

WHERE filters rows before grouping, while HAVING filters groups after aggregation. This difference is important when you want only customers whose total confirmed sales exceed a threshold.

Also remember

- COUNT(*) counts rows, while COUNT(column) counts non-null values. In optional Odoo fields, the difference can change...
- Aggregating after joins requires care. If a sale order is joined to both order lines and invoices, totals may be...

DB10 GROUP BY, HAVING, and Aggregates

Example and pitfalls

Monthly sales summary

The query filters confirmed orders first, groups by month and company, then keeps only meaningful sales groups.

```
SELECT
    date_trunc('month', so.date_order)::date AS month,
    so.company_id,
    COUNT(*) AS order_count,
    SUM(so.amount_total) AS total_amount
FROM sale_order so
WHERE so.state IN ('sale', 'done')
GROUP BY 1, 2
HAVING SUM(so.amount_total) > 10000
ORDER BY month DESC, total_amount DESC;
```

Common mistakes

- Selecting columns that are not part of GROUP BY and not aggregated.
- Using HAVING for simple row filters that belong in WHERE.
- Forgetting that COUNT(column) ignores NULL.
- Joining detail tables before aggregation and inflating totals.

DB10 GROUP BY, HAVING, and Aggregates

Extra patterns

Aggregate order

-

3 DB11 – DB15

PostgreSQL for Odoo · teaching block 3

DB11 Indexes: B-Tree, GIN, and Composite Design

Objectives, key points, and lab target

Objectives

- Explain how indexes speed up reads and slow down writes.
- Choose B-Tree, GIN, and composite indexes for common cases.
- Design indexes around Odoo domains and reporting queries.

Key points

- B-Tree is the default index type for most relational filters.
- Composite index column order should match query patterns.
- GIN is useful for jsonb, arrays, and specialized text search.
- Indexes speed reads but add write and storage cost.

Lab target

Choose one slow-looking domain from Odoo logs, write the SQL equivalent, run EXPLAIN ANALYZE, add an index in a test database, and compare the plan.

DB11 Indexes: B-Tree, GIN, and Composite Design

Teaching notes

Indexes are lookup structures that help PostgreSQL find rows without scanning an entire table. They are essential for large Odoo installations because tables such as `account_move_line`, `mail_message`, and `stock_move` can grow quickly.

B-Tree indexes are the default and support equality, range comparisons, ordering, and many common joins. They are usually the right first choice for ids, foreign keys, dates, states, and unique constraints.

Composite indexes include more than one column and should match real filter patterns. An index on `(company_id, state, date_order)` can help a domain that filters company and state before sorting or ranging by date.

Also remember

- GIN indexes are useful for containment and full-text style lookups, especially with `jsonb`, arrays, and trigram search...
- Every index has a write cost because PostgreSQL must maintain it during INSERT, UPDATE, and DELETE. Over-indexing an...

DB11 Indexes: B-Tree, GIN, and Composite Design

Example and pitfalls

Indexes for Odoo query patterns

The examples show B-Tree indexes for common domains and a GIN index for JSONB containment queries.

```
CREATE INDEX sale_order_company_state_date_idx  
ON sale_order (company_id, state, date_order DESC);
```

```
CREATE INDEX account_move_line_partner_date_idx  
ON account_move_line (partner_id, date);
```

```
CREATE INDEX integration_payload_payload_gin_idx  
ON integration_payload USING gin (payload);
```

```
EXPLAIN ANALYZE  
# ...
```

Common mistakes

- Adding indexes without measuring the query before and after.
- Creating many overlapping indexes on busy Odoo tables.
- Putting composite index columns in an order that does not match real filters.
- Expecting a normal B-Tree index to make leading-wildcard ILIKE searches fast.

DB12 Reading EXPLAIN ANALYZE

Objectives, key points, and lab target

Objectives

- Use EXPLAIN ANALYZE to inspect real query execution.
- Interpret scans, joins, row estimates, timing, and buffers.
- Turn query plans into practical optimization decisions.

Key points

- EXPLAIN ANALYZE executes the query.
- Compare estimated rows to actual rows.
- Look at buffers as well as timing.
- Sequential scans can be correct for small or broad queries.

Lab target

Run EXPLAIN (ANALYZE, BUFFERS) for a filtered query before and after ANALYZE, then document the scan type and row estimates.

DB12Reading EXPLAIN ANALYZE

Teaching notes

EXPLAIN shows PostgreSQL's planned execution strategy, while EXPLAIN ANALYZE runs the query and reports what actually happened. The ANALYZE form is more truthful but must be used carefully for writes because it executes them.

Plans are trees, and each node contributes work to its parent. Sequential scans, index scans, bitmap scans, nested loops, hash joins, and sorts all tell you how the optimizer chose to access and combine data.

Estimated rows versus actual rows is one of the most important signals. Large differences can mean stale statistics, correlated filters, missing indexes, or a query shape the planner cannot estimate well.

Also remember

- Timing tells you where time was spent, but buffers often explain why. A plan that reads many shared buffers may be...
- An index scan is not automatically good, and a sequential scan is not automatically bad. Scanning a small table or a...

DB12 Reading EXPLAIN ANALYZE

Example and pitfalls

Analyze a read query safely

The first command reports actual execution details, and the second refreshes statistics if estimates look stale.

```
EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
SELECT id, name, date_order
FROM sale_order
WHERE company_id = 1
      AND state = 'sale'
ORDER BY date_order DESC
LIMIT 20;

ANALYZE sale_order;
```

Common mistakes

- Running EXPLAIN ANALYZE on UPDATE or DELETE in production without wrapping and rolling...
- Assuming the largest cost number is always the real bottleneck.
- Ignoring row estimate errors.
- Adding an index because a sequential scan appears, even when the query returns most rows.

DB12 Reading EXPLAIN ANALYZE

Extra patterns

Plan reading checklist

- Which node reads the most rows?
- Where do estimates differ from actual rows?
- Is the query sorting, hashing, or looping more than expected?
- Are buffers high for a query that should be selective?

DB13 Backups with pg_dump

Objectives, key points, and lab target

Objectives

- Create logical backups with pg_dump.
- Choose custom, directory, and plain SQL dump formats.
- Prepare Odoo databases for consistent backup routines.

Key points

- Use custom format for flexible pg_restore workflows.
- Back up the Odoo filestore with the database.
- Automate backups and test restores.
- Protect dumps as sensitive production data.

Lab target

Create a custom-format dump of a training database and record the dump size, duration, and exact command used.

DB13 Backups with pg_dump

Teaching notes

pg_dump creates a logical backup by reading database objects and data. It is portable across machines and useful for Odoo migrations, staging refreshes, and point-in-time training snapshots.

The custom format is usually the best default for Odoo because it is compressed, flexible, and restorable with pg_restore. Plain SQL is readable but less flexible for selective restore and parallel jobs.

A consistent backup must include the PostgreSQL database and Odoo filestore. The database stores attachment metadata, but the binary files may live on disk under the Odoo data directory.

Also remember

- Backups should be automated and verified, not performed only when someone remembers. A backup that has never been...
- Large databases need attention to runtime, disk space, compression, and impact on the database server. pg_dump is...

DB13 Backups with pg_dump

Example and pitfalls

Custom-format Odoo backup

The database dump and filestore archive are both needed for a realistic Odoo recovery.

```
export DB=odoo17_prod
export BACKUP_DIR=/var/backups/odoo
sudo mkdir -p "$BACKUP_DIR"

pg_dump -h 127.0.0.1 -U odoo17 -F c -Z 6      -f "$BACKUP_DIR/odoo17_prod.dump"

sudo tar -C /var/lib/odoo/.local/share/Odoo/filestore -czf "$BACKUP_DIR/${DB}_filestore_$(date +%F).tar.gz" "
```

Common mistakes

- Backing up only PostgreSQL and forgetting the Odoo filestore.
- Using plain SQL dumps for every case and losing pg_restore flexibility.
- Keeping backups on the same disk as the database with no off-host copy.
- Never testing whether the backup can be restored.

```
-czf "$BACKUP_DIR/${DB}_filestore_$(date +%F).tar.gz" "
```

DB14 Restores with pg_restore

Objectives, key points, and lab target

Objectives

- Restore custom-format dumps safely.
- Use clean, create, jobs, and selective restore options.
- Validate an Odoo database after restore.

Key points

- Practice restore before a real incident.
- Prefer restoring into a fresh database for validation.
- Use --no-owner when target roles differ.
- Match the filestore directory name to the restored Odoo database.

Lab target

Restore a dump into a new training database, start Odoo against it, and open one attachment to prove the filestore is present.

DB14 Restores with pg_restore

Teaching notes

Restore practice is where backup confidence becomes real. A team that restores regularly can estimate recovery time, detect missing filestore data, and find permission problems before an emergency.

`pg_restore` works with custom and directory-format dumps. It can create objects, clean existing objects, run parallel jobs, and restore selected schemas or tables when the dump format supports it.

Restoring into a fresh database is usually safer than overwriting an existing one. It lets you validate the result before swapping names, updating configuration, or pointing Odoo at the restored database.

Also remember

- Ownership and role availability matter during restore. If the dump references owners that do not exist on the target...
- For Odoo, restore the filestore directory with the exact database name expected by the instance. If the database is...

DB14 Restores with pg_restore

Example and pitfalls

Restore a custom dump to a test database

The restore targets a separate database and performs a simple Odoo metadata check afterward.

```
createdb -h 127.0.0.1 -U odoo17 odoo17_restore_test  
pg_restore -h 127.0.0.1 -U odoo17 --dbname=odoo17_  
psql -h 127.0.0.1 -U odoo17 -d odoo17_restore_test
```

Common mistakes

- Restoring over the only available copy of a database.
- Ignoring missing target roles and ownership errors.
- Forgetting to restore or rename the filestore.
- Declaring success after pg_restore exits without checking Odoo startup.

```
-c "SELECT latest_version FROM ir_module_module WHERE name
```

DB15 postgresql.conf and pg_hba.conf

Objectives, key points, and lab target

Objectives

- Locate and edit the main PostgreSQL configuration files.
- Configure listening addresses, memory settings, logging, and authentication.
- Reload or restart PostgreSQL safely after configuration changes.

Key points

- postgresql.conf tunes server behavior.
- pg_hba.conf controls connection authentication.
- pg_hba.conf rule order matters.
- Reload when possible, restart when required.

Lab target

Find config_file and hba_file with SHOW commands, add a localhost rule for your Odoo role in a lab environment, reload, and reconnect.

DB15 postgresql.conf and pg_hba.conf

Teaching notes

postgresql.conf controls server behavior such as listening addresses, memory, logging, autovacuum, and connection limits. pg_hba.conf controls who can connect, from where, to which database, and by which authentication method.

Configuration files are cluster-specific on Ubuntu. The common path includes the PostgreSQL major version, such as /etc/postgresql/15/main, so always confirm the active cluster before editing.

listen_addresses determines the network interfaces PostgreSQL listens on. Binding to localhost is safest for single-host Odoo; binding to all addresses requires firewalling and strong authentication rules.

Also remember

- pg_hba.conf is evaluated top to bottom, and the first matching rule wins. A permissive rule above a strict rule can...
- Many changes only need a reload, while others require a restart. Reloading is less disruptive, but you must know...

DB15 postgresql.conf and pg_hba.conf

Example and pitfalls

Odoo-friendly local configuration snippets

The snippets keep PostgreSQL local to the application host and enable slow-query visibility for tuning.

```
-- Locate files
SHOW config_file;
SHOW hba_file;

-- postgresql.conf examples
listen_addresses = '127.0.0.1'
max_connections = 200
shared_buffers = 1GB
log_min_duration_statement = 500

# ...
```

Common mistakes

- Editing the wrong versioned cluster directory.
- Adding a strict pg_hba.conf rule below a broader rule that already matches.
- Using trust authentication on shared or production machines.
- Restarting the service for every change without knowing whether reload is enough.

4 DB16 – DB16

PostgreSQL for Odoo · teaching block 4

DB16 Locks, Connections, Terminating Queries, and Odoo Backup Habits

Objectives, key points, and lab target

Objectives

- Inspect active connections and blocking locks.
- Terminate unsafe or stuck sessions carefully.
- Apply Odoo-specific database habits and backup strategy.

Key points

- Use `pg_stat_activity` to understand sessions before acting.
- Find blockers, not just blocked queries.
- Cancel or terminate with intent and an audit trail.
- Odoo backups must include both database and filestore.
- Restore drills are part of the backup strategy.

Lab target

Open two psql sessions, create a blocking transaction in one, observe it from `pg_stat_activity` in the other, then cancel or roll back safely.

DB16 Locks, Connections, Terminating Queries, and Odoo Backup Habits

Teaching notes

Every PostgreSQL connection uses server resources, and every transaction can hold locks. Odoo workers, cron jobs, imports, reports, and shell sessions can all contribute to lock contention.

Locks are normal; blocking is the operational concern. A short row lock during a write is expected, while an idle transaction holding a table lock can stop module upgrades or business workflows.

`pg_stat_activity` shows what sessions are doing, how long they have been running, and whether they are waiting. Combined with `pg_locks`, it helps identify blockers rather than only victims.

Also remember

- Terminating a backend is a production action, not a reflex. Prefer canceling a query when possible, communicate with...
- Odoo-specific database habits include avoiding direct writes to business tables, stopping Odoo during risky restores,...

DB16 Locks, Connections, Terminating Queries, and Odoo Backup Habits

Example and pitfalls

Inspect and control active sessions

Inspect first, cancel when possible, and terminate only when the risk of leaving the session running is greater than killing it.

```
SELECT pid, username, datname, state, wait_event_type,  
       now() - query_start AS runtime,  
       left(query, 120) AS query  
FROM pg_stat_activity  
WHERE datname = 'odoo17_prod'  
ORDER BY runtime DESC NULLS LAST;
```

```
SELECT pg_cancel_backend(12345);  
SELECT pg_terminate_backend(12345);
```

Common mistakes

- Killing sessions without checking whether they are blockers or victims.
- Leaving psql transactions open with state idle in transaction.
- Refreshing staging from production while Odoo cron jobs are active.
- Backing up databases and filestores on different schedules so they no longer match.

Simple Odoo backup policy outline

Daily:

- pg_dump custom format for each production database
- tar and compress matching filestore directories
- upload encrypted artifacts to off-host storage

Weekly:

- restore latest backup to an isolated server

...

DB16 Locks, Connections, Terminating Queries, and Odoo Backup Habits

Extra patterns

Senior rule

During an incident, identify the blocking session and business impact before terminating anything.

Σ You should now be able to...

Section wrap-up

- DB01 Install PostgreSQL on Ubuntu
- DB02 psql CLI, Users, Roles, and Grants
- DB03 Create and Manage Databases
- DB04 PostgreSQL Data Types for Odoo Developers
- DB05 DDL: CREATE, ALTER, and DROP
- DB06 Primary Keys, Foreign Keys, and Constraints
- DB07 CRUD: INSERT, SELECT, UPDATE, and DELETE
- DB08 Filtering with WHERE, LIKE, ILIKE, IN, and BETWEEN
- Complete every lab before the next section
- Keep a command/error journal
- Prefer disposable databases for experiments
- Re-read pitfalls before production changes

PostgreSQL

SQL

Indexes

Backup

NEXT STEP

Complete the section labs

Open the next presentation deck

Section 03 · Database

